

LAPORAN PRAKTIKUM

STRATEGI ALGORITMA

PRAKTIKUM 1: KOMPLEKSITAS ALGORITMA

I. DEFINISI

1.1 Kompleksitas Algoritma

Kompleksitas algoritma adalah ukuran efisiensi suatu algoritma dilihat dari sumber daya yang dibutuhkan, yaitu waktu dan memori. Kompleksitas Waktu $T(n)$ mengukur jumlah operasi sebagai fungsi ukuran masukan n , sedangkan Kompleksitas Ruang $S(n)$ mengukur penggunaan memori. Analisis dilakukan pada tiga kondisi: best case (kondisi terbaik), worst case (kondisi terburuk), dan average case (kondisi rata-rata).

1.2 Notasi Big-O

Notasi Big-O menyatakan batas atas pertumbuhan kompleksitas suatu algoritma, mengabaikan konstanta dan suku tidak dominan. Hierarki kompleksitas dari yang paling efisien:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

1.3 Selection Sort

Definisi: Algoritma pengurutan yang bekerja dengan mencari elemen terkecil dari bagian array yang belum terurut, lalu menukarnya ke posisi yang tepat.

Cara Kerja: Setiap pass ke- i mencari elemen minimum dari $A[i..n-1]$ dan menukarnya dengan $A[i]$. Setelah pass ke- i , elemen $A[0..i]$ sudah pada posisi akhirnya.

Fungsi: Mengurutkan array secara ascending dengan pendekatan pemilihan elemen minimum.

Kelebihan: Sederhana, jumlah pertukaran minimal (maksimal $n-1$ kali), dan in-place.

Kekurangan: Kompleksitas $O(n^2)$ di semua kasus, tidak stabil, dan tidak adaptif.

1.4 Insertion Sort

Definisi: Algoritma pengurutan yang menyisipkan setiap elemen ke posisi yang tepat dalam bagian array yang sudah terurut, mirip cara menyortir kartu.

Cara Kerja: Mulai dari indeks $i=1$, setiap elemen $key=A[i]$ dibandingkan dengan elemen di kirinya. Elemen yang lebih besar digeser ke kanan hingga posisi yang tepat ditemukan.

Fungsi: Mengurutkan array secara ascending dengan pendekatan penyisipan elemen.

Kelebihan: Efisien untuk data kecil atau hampir terurut (best case $O(n)$), stabil, adaptif, dan in-place.

Kekurangan: Worst case $O(n^2)$ dan banyak operasi pergeseran pada data acak berukuran besar.

II. PRE TEST

Pertanyaan 1: Apa alat ukur sebuah algoritma dikatakan efisien menjadi solusi suatu permasalahan?

Jawaban: Alat ukur efisiensi algoritma ada dua: Kompleksitas Waktu $T(n)$ yang mengukur jumlah operasi komputasi sebagai fungsi ukuran masukan n , dan Kompleksitas Ruang $S(n)$ yang mengukur kebutuhan memori. Keduanya bersifat abstrak dan independen dari perangkat keras maupun compiler sehingga dapat digunakan sebagai acuan universal dalam membandingkan algoritma.

Pertanyaan 2: Mengapa kita perlu algoritma yang efisien?

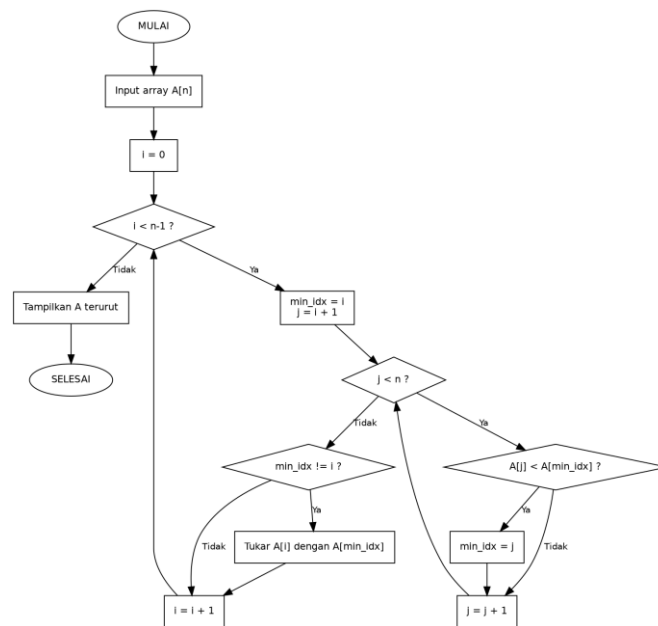
Jawaban: Algoritma yang efisien diperlukan karena:

1. Selisih kompleksitas sangat terasa pada data besar. Untuk $n = 1.000.000$, algoritma $O(n)$ butuh 10^6 operasi sedangkan $O(n^2)$ butuh 10^{12} operasi.
2. Menghemat sumber daya komputasi berupa waktu proses, memori, dan energi.
3. Memungkinkan sistem yang skalabel karena performa tetap baik meskipun data bertambah.

III. ANALISIS PRAKTIKUM

3.1 Langkah Praktikum: Selection Sort

Flowchart



Gambar 3.1 Flowchart Algoritma Selection Sort

Source Code Python

```
def selection_sort(arr):
    """
    Mengurutkan array dengan algoritma Selection Sort.
    Setiap pass mencari elemen terkecil dari sisa array
    lalu menukarnya ke posisi yang tepat.
    """
    n = len(arr)
    total_comparisons = 0
    total_swaps = 0

    print(f"Array awal : {arr}")
    print("-" * 55)

    for i in range(n - 1):
        # Asumsikan elemen terkecil ada di posisi i
        min_idx = i
        pass_comparisons = 0
        pass_swaps = 0

        # Cari elemen terkecil dari i+1 hingga akhir array
        for j in range(i + 1, n):
            pass_comparisons += 1
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Tukar elemen terkecil ke posisi i (jika berbeda)
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
            pass_swaps += 1

        total_comparisons += pass_comparisons
        total_swaps += pass_swaps

        print(f"Pass {i + 1:2d} | Array: {arr} | "
              f"Perbandingan: {pass_comparisons:2d} | "
              f"Pertukaran: {pass_swaps}")

    print("-" * 55)
    print(f"Array akhir      : {arr}")
    print(f"Total perbandingan  : {total_comparisons}")
    print(f"Total pertukaran    : {total_swaps}")
```

```
return arr
```

Analisis

Jawaban 1: Jumlah operasi perbandingan untuk setiap pass ke-i

Pada pass ke-i (i dari 0 sampai n-2), algoritma mencari elemen minimum dari posisi i+1 hingga n-1. Jumlah perbandingan pada setiap pass:

Pass ke-i	Rentang Pencarian	Perbandingan
0	A[1] hingga A[4]	4
1	A[2] hingga A[4]	3
2	A[3] hingga A[4]	2
3	A[4] hingga A[4]	1

Jawaban 2: Jumlah operasi pertukaran elemen

Setiap pass melakukan paling banyak 1 kali pertukaran, hanya apabila elemen minimum (min_idx) bukan berada di posisi i. Pertukaran minimum 0 kali (array sudah terurut) dan maksimum n-1 kali. Pada Kasus Uji 1 dengan n=5, total pertukaran = 3 dari 4 pass.

Jawaban 3: Derivasi kompleksitas waktu

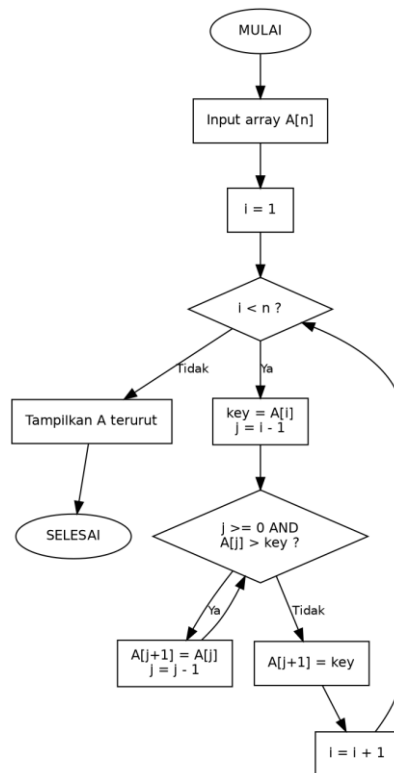
Total perbandingan dijumlahkan dari seluruh pass:

$$T(n) = (n-1) + (n-2) + \dots + 1 = n(n-1) / 2$$

Untuk n=5: $T(5) = 5 \times 4 / 2 = 10$, sesuai output program. Suku dominan adalah n^2 , sehingga kompleksitas waktu Selection Sort adalah $O(n^2)$.

3.2 Post Test: Insertion Sort

Flowchart



Gambar 3.2 Flowchart Algoritma Insertion Sort

Source Code Python

```

def insertion_sort(arr):
    """
    Mengurutkan array dengan algoritma Insertion Sort.
    Setiap elemen disisipkan ke posisi yang tepat
    di dalam bagian array yang sudah terurut.
    """
    n = len(arr)
    total_comparisons = 0
    total_shifts = 0

    print(f"Array awal : {arr}")
    print("-" * 60)

    for i in range(1, n):
        key = arr[i]
        j = i - 1
        pass_comparisons = 0
        pass_shifts = 0

        # Geser elemen yang lebih besar dari key ke kanan
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
            pass_comparisons += 1
            pass_shifts += 1

        # Hitung perbandingan terakhir (kondisi berhenti)
        if j >= 0:
            pass_comparisons += 1

        # Sisipkan key ke posisi yang tepat
        arr[j + 1] = key

        total_comparisons += pass_comparisons
        total_shifts += pass_shifts

    print(f"Pass {i:2d} | key={key:3d} | Array: {arr} | ")
  
```

```

        f"Perbandingan: {pass_comparisons} | "
        f"Pergeseran: {pass_shifts}")

    print("-" * 60)
    print(f"Array akhir      : {arr}")
    print(f"Total perbandingan : {total_comparisons}")
    print(f"Total pergeseran    : {total_shifts}")

    return arr

```

Analisis

Jawaban 1: Kompleksitas waktu Insertion Sort

- **Best Case $O(n)$:** Array sudah terurut. Setiap elemen hanya dibandingkan sekali, tidak ada pergeseran. $T(n) = n-1$.
- **Worst Case $O(n^2)$:** Array terurut terbalik. Elemen ke- i harus digeser i kali. $T(n) = 1+2+\dots+(n-1) = n(n-1)/2$.
- **Average Case $O(n^2)$:** Rata-rata elemen ke- i melakukan $i/2$ perbandingan. $T(n) \approx n(n-1)/4$.

Jawaban 2: Perbandingan Selection Sort vs Insertion Sort

Aspek	Insertion Sort	Selection Sort
Best Case	$O(n)$ — adaptif	$O(n^2)$ — tidak adaptif
Worst Case	$O(n^2)$	$O(n^2)$
Average Case	$O(n^2)$	$O(n^2)$
Pertukaran/Geser	Banyak (pergeseran)	Sedikit (maks. $n-1$)
Stabilitas	Stabil	Tidak Stabil
Adaptivitas	Adaptif	Tidak Adaptif

Kesimpulan: Insertion Sort lebih mangkus pada data yang hampir terurut karena bersifat adaptif dengan best case $O(n)$. Selection Sort lebih unggul dalam minimasi pertukaran (maks. $n-1$ swap), cocok ketika operasi swap memiliki biaya mahal. Pada data acak berukuran besar keduanya setara di $O(n^2)$.

IV. IMPLEMENTASI NYATA

Selection Sort dan Insertion Sort banyak diimplementasikan pada sistem dengan dataset kecil, seperti sistem pengolahan nilai akademik, antrian transaksi sederhana, atau pengurutan log pada embedded system. Dua jurnal berikut mendukung relevansi penggunaan kedua algoritma ini dalam praktik.

[1] **Penulis:** Karunanidhi, G. (2025)

Judul: *"A Systematic Analysis on Performance and Computational Complexity of Sorting Algorithms"*

Publikasi: Discover Computing, Vol. 28, No. 1, Hal. 250. Springer Nature. DOI: 10.1007/s10791-025-09724-w

Intisari: Review sistematis terhadap 12 teknik sorting klasik menggunakan analisis empiris pada MATLAB dengan ukuran data 100 hingga 100.000 elemen. Hasil menunjukkan bahwa selection sort mengalami penurunan performa signifikan pada data besar karena kompleksitas $O(n^2)$ yang konsisten di semua kasus. Sebaliknya, insertion sort terbukti lebih efisien untuk data berukuran kecil berkat sifat adaptifnya.

Link: <https://link.springer.com/article/10.1007/s10791-025-09724-w>

[2] **Penulis:** Halili, F. dan Dika, E. (2021)

Judul: *"Results of Comparison Between Different Sorting Algorithms"*

Publikasi: IBU International Journal of Technical and Natural Sciences, Vol. 2, Hal. 39-52. International Balkan University Press.

Intisari: Studi komparatif lima algoritma pengurutan berdasarkan waktu eksekusi menggunakan C++. Hasil empiris menunjukkan bahwa Insertion Sort merupakan pilihan yang baik untuk dataset berukuran kecil hingga beberapa ribu elemen karena sifat adaptif (best case $O(n)$) dan kemudahan implementasinya. Insertion Sort tidak disarankan untuk list dengan lebih dari beberapa ribu elemen karena kompleksitas kuadratnya.

Link: <https://ijtns.ibupress.com/uploads/pdf/22.pdf>

Kelebihan di konteks dataset kecil:

- Implementasi sederhana dan mudah dipelihara.
- Tidak memerlukan memori tambahan (in-place).
- Insertion Sort efisien untuk data yang ditambahkan secara bertahap (inkremental).

Kekurangan di konteks dataset kecil:

- Tidak cocok untuk $n > 10.000$ karena kompleksitas $O(n^2)$ menjadi bottleneck.

- Untuk kebutuhan skala besar, disarankan beralih ke Merge Sort atau Quick Sort dengan kompleksitas $O(n \log n)$.

PRAKTIKUM 2: ALGORITMA BRUTE FORCE

I. DEFINISI

1.1 Algoritma Brute Force

Definisi: Brute Force adalah pendekatan lempang (straightforward) untuk memecahkan suatu persoalan dengan cara yang paling langsung dan jelas (obvious way), tanpa memanfaatkan struktur khusus dari masalah yang diselesaikan.

Cara Kerja: Setiap kemungkinan solusi dieksplorasi satu per satu secara sistematis hingga solusi yang benar ditemukan atau seluruh kemungkinan habis diperiksa.

Fungsi: Digunakan sebagai solusi dasar (baseline) untuk masalah yang belum memiliki algoritma khusus, atau ketika ukuran masukan cukup kecil sehingga kompleksitas tinggi masih dapat ditoleransi.

Kelebihan: Mudah dipahami dan diimplementasikan, selalu menghasilkan solusi yang benar apabila solusi memang ada, dan tidak memerlukan analisis matematis yang rumit.

Kekurangan: Kompleksitas waktu umumnya tinggi ($O(n^2)$ atau lebih), tidak efisien untuk data besar, dan tidak memanfaatkan struktur masalah untuk mempercepat pencarian.

1.2 Perpangkatan

Menghitung a^n ($a > 0$, n bilangan bulat tak-negatif) dengan cara mengalikan 1 dengan a sebanyak n kali secara berurutan. Ciri Brute Force: tidak menggunakan teknik fast exponentiation atau divide and conquer, melainkan perkalian berulang satu per satu.

1.3 Pencarian Elemen Terbesar

Mencari elemen terbesar dari array berisi n bilangan bulat dengan membandingkan setiap elemen dengan nilai maksimum sementara. Ciri Brute Force: setiap elemen diperiksa satu per satu tanpa pengecualian. Kompleksitas $T(n) = n - 1$ perbandingan, sehingga $O(n)$.

1.4 Perkalian Dua Matriks

Menghitung hasil perkalian dua matriks A dan B berukuran $n \times n$ dengan menghitung setiap elemen $C[i][j]$ satu per satu melalui tiga perulangan bersarang (i, j, k). Ciri Brute Force: tidak ada optimasi seperti Strassen. Kompleksitas $O(n^3)$.

1.5 Uji Bilangan Prima

Definisi: Bilangan prima adalah bilangan yang hanya habis dibagi oleh 1 dan dirinya sendiri.

Ciri Brute Force: membagi x dengan setiap bilangan dari 2 sampai $\sqrt{x}$ satu per satu. Jika tidak ada yang habis membagi, maka x adalah bilangan prima. Kompleksitas worst case $O(\sqrt{n})$.

II. PRE TEST

Pertanyaan 1: Jelaskan apa ciri khas dari penyelesaian persoalan dengan algoritma Brute Force!

Jawaban: Ciri khas Brute Force adalah penyelesaian masalah secara langsung, sederhana, dan jelas (straightforward/obvious way) tanpa memanfaatkan struktur atau sifat khusus dari masalah. Setiap kemungkinan diperiksa satu per satu secara menyeluruh. Pendekatan ini menjamin solusi yang benar namun umumnya menghasilkan kompleksitas yang tinggi karena tidak ada optimasi.

Pertanyaan 2: Jelaskan ciri khas algoritma Brute Force pada contoh berikut:

a. Algoritma Perpangkatan: Brute Force terlihat dari cara menghitung a^n dengan mengalikan a sebanyak n kali secara berulang (hasil = hasil * a), tanpa memanfaatkan teknik seperti fast exponentiation. Kompleksitas $O(n)$.

b. Algoritma Pencarian Elemen Terbesar: Brute Force terlihat dari perbandingan setiap elemen $A[k]$ dengan nilai maks satu per satu dari awal hingga akhir array. Tidak ada strategi pemangkasan. Kompleksitas $O(n)$.

c. Algoritma Perkalian Dua Matriks: Brute Force terlihat dari tiga perulangan bersarang (i, j, k) yang menghitung setiap elemen $C[i][j]$ satu per satu tanpa optimasi. Kompleksitas $O(n^3)$.

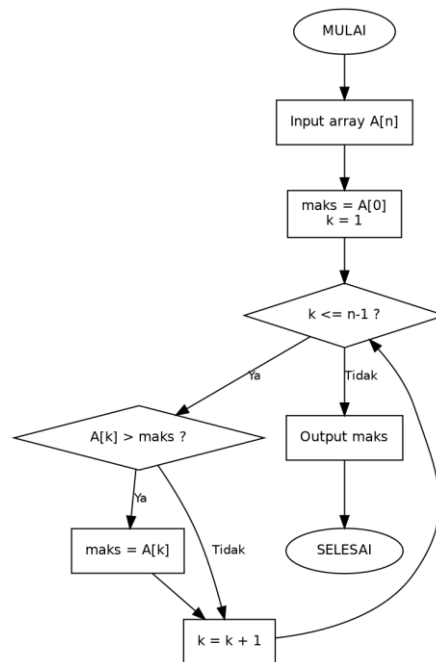
d. Algoritma Uji Bilangan Prima: Brute Force terlihat dari pembagian x dengan setiap bilangan dari 2 hingga $\sqrt{x}$ secara berurutan. Tidak ada penggunaan sifat bilangan prima untuk mempercepat. Kompleksitas $O(\sqrt{n})$.

III. ANALISIS PRAKTIKUM

3.1 Langkah Praktikum

(a) Pencarian Elemen Terbesar

Flowchart



Gambar 3.1 Flowchart Algoritma Pencarian Elemen Terbesar

Source Code Python

```
def cari_element_terbesar(arr):
    """
    Mencari elemen terbesar dalam array menggunakan Brute Force.
    Strategi: bandingkan setiap elemen dengan nilai maksimum sementara.
    Kompleksitas: T(n) = n - 1 perbandingan -> O(n)
    """
    n = len(arr)
    maks = arr[0]          # asumsikan elemen pertama adalah terbesar
    idx_maks = 0

    print(f"Array          : {arr}")
    print(f"Inisialisasi : maks = {maks} (indeks 0)")
    print("-" * 55)

    for k in range(1, n):
        # bandingkan setiap elemen dengan nilai maks saat ini
        if arr[k] > maks:
            maks = arr[k]
            idx_maks = k
            status = f"UPDATE maks = {maks}"
        else:
            status = "tidak ada perubahan"

        print(f"k={k} | A[k]={arr[k]:4d} | maks={maks:4d} | {status}")

    print("-" * 55)
    print(f"Elemen terbesar : {maks} (indeks {idx_maks})")
    print(f"Jumlah perbandingan : {n - 1} --> T(n) = n - 1 = O(n)")
    return maks
```

```
# =====
# Main Program
# =====
if __name__ == "__main__":

    # Kasus Uji 1: Array acak
    data1 = [3, 7, 1, 9, 4, 6, 2]
    print("=" * 55)
    print("  PENCARIAN ELEMEN TERBESAR - Kasus Uji 1 (acak)")
    print("=" * 55)
    cari_element_terbesar(data1)

    # Kasus Uji 2: Terbesar di awal (best case perbaruan)
    data2 = [15, 3, 8, 1, 10]
    print("\n" + "=" * 55)
    print("  PENCARIAN ELEMEN TERBESAR - Kasus Uji 2 (terbesar di awal)")
    print("=" * 55)
    cari_element_terbesar(data2)

    # Kasus Uji 3: Terbesar di akhir (worst case perbaruan)
    data3 = [2, 5, 8, 11, 20]
    print("\n" + "=" * 55)
    print("  PENCARIAN ELEMEN TERBESAR - Kasus Uji 3 (terbesar di akhir)")
    print("=" * 55)
    cari_element_terbesar(data3)

# =====
# Contoh Output:
# =====
# PENCARIAN ELEMEN TERBESAR - Kasus Uji 1 (acak)
# =====
# Array      : [3, 7, 1, 9, 4, 6, 2]
# Inisialisasi : maks = 3 (indeks 0)
# -----
# k=1 | A[k]= 7 | maks= 7 | UPDATE maks = 7
# k=2 | A[k]= 1 | maks= 7 | tidak ada perubahan
# k=3 | A[k]= 9 | maks= 9 | UPDATE maks = 9
# k=4 | A[k]= 4 | maks= 9 | tidak ada perubahan
# k=5 | A[k]= 6 | maks= 9 | tidak ada perubahan
# k=6 | A[k]= 2 | maks= 9 | tidak ada perubahan
# -----
# Elemen terbesar      : 9 (indeks 3)
# Jumlah perbandingan : 6 --> T(n) = n - 1 = O(n)
```

Analisis Output

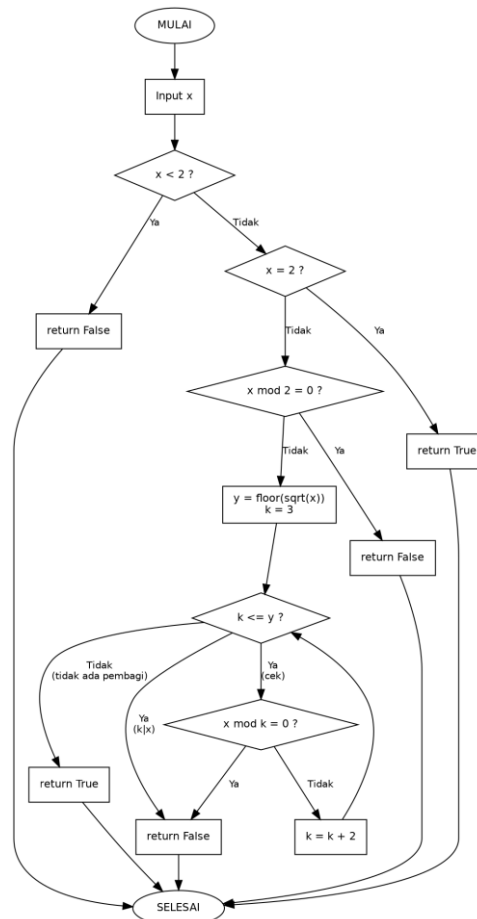
Program diuji dengan tiga kasus:

- **Kasus 1 (acak)** Array [3, 7, 1, 9, 4, 6, 2]: elemen terbesar = 9 ditemukan pada indeks 3. Jumlah perbandingan = 6 ($n-1 = 7-1$). Terjadi dua kali UPDATE maks, yaitu saat $k=1$ (nilai 7) dan $k=3$ (nilai 9).
- **Kasus 2 (terbesar di awal)** Array [15, 3, 8, 1, 10]: elemen terbesar = 15 langsung ditemukan di inisialisasi. Tidak ada UPDATE maks selama iterasi. Jumlah perbandingan tetap = 4 ($n-1$).
- **Kasus 3 (terbesar di akhir)** Array [2, 5, 8, 11, 20]: maks diperbarui di setiap iterasi karena data terurut menaik. Jumlah perbandingan = 4 ($n-1$).

Ketiga kasus menunjukkan bahwa jumlah perbandingan selalu $n-1$, tidak bergantung pada posisi elemen terbesar. Ini membuktikan $T(n) = n - 1$ sehingga $O(n)$.

(b) Uji Bilangan Prima

Flowchart



Gambar 3.2 Flowchart Algoritma Uji Bilangan Prima

Source Code Python

```

def is_prima(x):
    """
    Menguji apakah x adalah bilangan prima menggunakan Brute Force.
    Strategi: bagi x dengan setiap bilangan dari 2 sampai sqrt(x).
    Jika tidak ada yang habis membagi, maka x adalah bilangan prima.

    Kompleksitas:
        Worst case : T(n) = sqrt(n) - 1 --> O(sqrt(n))
        Best case  : T(n) = 1          --> O(1)   (genap atau < 2)
    """
    if x < 2:
        return False      # 1 dan bilangan negatif bukan prima
    if x == 2:
        return True       # 2 adalah prima, kasus khusus
    if x % 2 == 0:
        return False      # bilangan genap selain 2 bukan prima

    y = int(math.sqrt(x)) # batas atas pembagi
    k = 3                 # mulai dari 3, langkah 2 (hanya bilangan ganjil)

```

```
while k <= y:
    if x % k == 0:
        return False      # ditemukan pembagi, bukan prima
    k += 2

return True                # tidak ada pembagi ditemukan, x adalah prima
```

Analisis Output

Program mencetak semua bilangan prima dari 1 hingga 50: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47] sebanyak 15 bilangan.

Pengujian verbose pada beberapa angka:

- **x = 17** (prima): $\text{sqrt}(17) = 4$, hanya 1 langkah ($17 \% 3 = 2$), langsung dinyatakan prima.
- **x = 18** (bukan prima): langsung dieliminasi di pengecekan bilangan genap tanpa masuk loop.
- **x = 37** (prima): $\text{sqrt}(37) = 6$, 2 langkah ($\% 3$ dan $\% 5$), dinyatakan prima.
- **x = 49** (bukan prima): $\text{sqrt}(49) = 7$, pada langkah ke-3 ditemukan $49 \% 7 = 0$, bukan prima.

3.2 Post Test

Jawaban 1: Kompilasi dan eksekusi program

Program dijalankan menggunakan Python 3 dan menghasilkan output yang sesuai dengan pseudocode modul. Pencarian elemen terbesar menghasilkan hasil yang konsisten dengan $T(n) = n-1$ perbandingan di setiap kasus uji, sedangkan uji bilangan prima berhasil mengidentifikasi 15 bilangan prima dalam rentang 1 hingga 50 dengan benar.

Jawaban 2: Implementasi Python pencarian elemen terbesar

Implementasi Python dari pseudocode modul terdapat pada file `pencarian_terbesar.py` dengan fungsi `cari_element_terbesar(arr)`. Kode mengonversi prosedur `CariElemenTerbesar` dari pseudocode ke Python 3 dengan penambahan pelacakan jumlah perbandingan dan tampilan status tiap iterasi (lihat source code pada 3.1a).

Jawaban 3: Kompleksitas algoritma uji bilangan prima

Analisis kompleksitas dilakukan berdasarkan jumlah iterasi pembagian:

- **Best Case O(1):** Terjadi ketika $x < 2$, $x = 2$, atau x adalah bilangan genap ($x \% 2 = 0$). Tidak masuk loop, hanya 1 operasi pengecekan.
- **Worst Case O(sqrt(n)):** Terjadi ketika x adalah bilangan prima. Loop berjalan dari $k=3$ hingga $k=\sqrt{x}$ dengan langkah 2. Jumlah iterasi maksimum = $(\sqrt{x}) - 3)/2 + 1$, yang bersifat $O(\sqrt{n})$.

Bilangan x	$\sqrt{x}$	Iterasi Maksimum	Hasil
17	4	1	Prima
37	6	2	Prima
97	9	4	Prima
49	7	3	Bukan Prima ($49 \% 7 = 0$)

Karena iterasi dibatasi oleh $\sqrt{x}$, maka kompleksitas waktu uji bilangan prima adalah $T(n) = O(\sqrt{n})$. Ini jauh lebih efisien dibandingkan pendekatan Brute Force murni yang mencoba semua pembagi dari 2 hingga $x-1$ dengan kompleksitas $O(n)$.

IV. IMPLEMENTASI NYATA

Algoritma Brute Force seperti pencarian elemen terbesar dan uji bilangan prima digunakan secara nyata dalam berbagai bidang, mulai dari kriptografi hingga sistem keamanan jaringan. Dua jurnal berikut mendukung relevansinya.

[1] **Penulis:** Shifa Thamkin, S., Raksha, H.M., dan Mohammed Rafi (2024)

Judul: *"Review Paper on Data Recovery Using Brute Force Algorithm"*

Publikasi: Journal of Computer Technology and Applications (JoCTA), STM Journals, Vol. 15, No. 3, 2024.

Intisari: Studi ini membahas penerapan Brute Force dalam pemulihan data digital, khususnya pada proses file carving dan password recovery. Brute Force menjadi metode andalan karena kemampuannya memeriksa seluruh kemungkinan secara menyeluruh tanpa memerlukan informasi awal tentang struktur data. Relevan dengan uji bilangan prima yang juga memeriksa seluruh pembagi secara sistematis.

Link: <https://journals.stmjournals.com/jocta/article=2024/view=177277/>

[2] **Penulis:** Baur, K. et al. (2023)

Judul: *"Comparing and Reviewing Modern Primality Tests"*

Publikasi: ResearchGate, 2023. Tersedia di:
<https://www.researchgate.net/publication/369685477>

Intisari: Makalah ini membandingkan berbagai algoritma uji bilangan prima mulai dari Brute Force trial division ($O(\sqrt{n})$) hingga algoritma modern seperti Miller-Rabin dan AKS. Brute Force digunakan sebagai baseline referensi karena kesederhanaan dan ketepatannya. Hasilnya menunjukkan bahwa Brute Force masih relevan untuk bilangan kecil, namun tidak skalabel untuk bilangan kriptografis berukuran besar (ratusan digit).

Link:

https://www.researchgate.net/publication/369685477_Comparing_and_Reviewing_Modern_Primality_Tests

Kelebihan dalam konteks nyata:

- Implementasi sederhana dan mudah diverifikasi kebenarannya.
- Cocok sebagai baseline perbandingan sebelum menggunakan algoritma yang lebih kompleks.
- Uji bilangan prima $O(\sqrt{n})$ sudah cukup efisien untuk kebutuhan pembuatan kunci pada sistem kriptografi skala kecil.

Kekurangan dalam konteks nyata:

- Tidak skalabel: untuk bilangan prima kriptografis (ratusan digit), kompleksitas $O(\sqrt{n})$ masih terlalu lambat dan perlu diganti algoritma probabilistik seperti Miller-Rabin.
- Pencarian elemen terbesar $O(n)$ memang sudah optimal untuk kasus linear, namun tidak berlaku jika data terstruktur (misal sudah terurut) yang memungkinkan pencarian lebih cepat.